Lecture 28

Ant Colony Optimisation

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool

Lecture goals

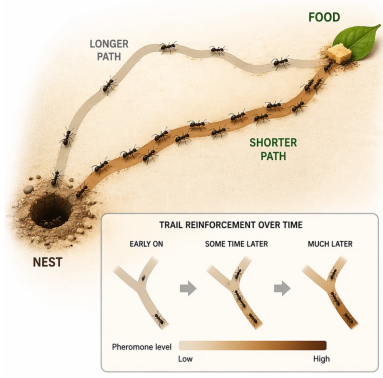
- Introduce **Ant Colony Optimisation (ACO)** as a method for constructive combinatorial search.
- Explain how **stigmergy** and **pheromone trails** provide a form of environmental memory.
- Understand the core ACO loop: **construct**, **evaluate**, **reinforce**, and **evaporate**.
- Use the **Traveling Salesman Problem** as a running example.

From real ants to artificial search

- Real ants can collectively discover short paths between nest and food.
- They do this without global planning or explicit route maps.
- The key mechanism is **trail reinforcement**: useful paths attract more future traffic.

Key idea

Use indirect communication through the environment to guide combinatorial search.



Shortest path intuition

- Ants deposit pheromone while moving.
- Shorter paths are traversed more quickly.
- Faster round trips lead to more frequent reinforcement.
- More pheromone attracts more ants, which reinforces the path even further.

Autocatalytic effect

A small initial advantage can be amplified into a strong collective preference.

What ACO changes

- We do not literally simulate ants walking around a forest.
- Instead, we place artificial ants on a **graph of decisions**.
- Ants incrementally build candidate solutions.
- Good solutions increase pheromone on useful components of that graph.

Interpretation

ACO is a framework for **learning which solution components are worth reusing**.

ACO in four steps

1. **Construct** a solution step by step.
2. **Evaluate** the quality of the completed solution.
3. **Reinforce** components used by good solutions.
4. **Evaporate** old pheromone to avoid lock-in.

Why this matters

ACO searches by maintaining and updating a **population-level memory over decisions**.

How ACO differs from PSO

PSO

- continuous search space
- particles move in state space
- memory stored in particles

ACO

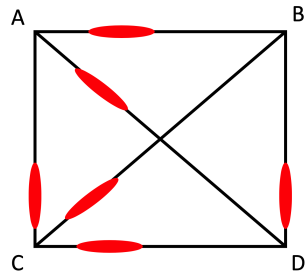
- combinatorial search space
- ants build solutions incrementally
- memory stored in the environment

Construction graphs and partial solutions

- A problem is represented as a graph of possible choices.
- A partial solution corresponds to a path through that graph.
- At each step, an ant chooses a feasible next component.
- Constraints determine which choices are still legal.

Key perspective

ACO is especially natural when solutions must be **built one decision at a time**.



4-city example.

The state transition rule

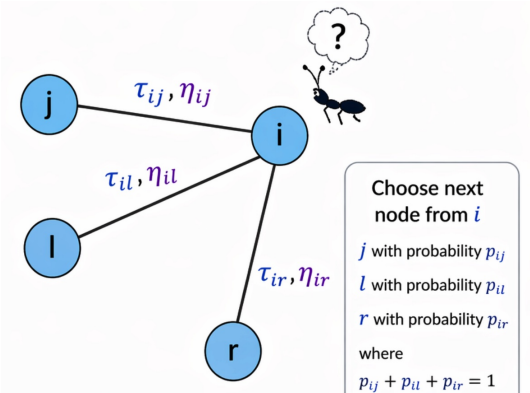
A common choice rule is:

$$p_{ij} = \frac{\tau_{ij}^{\alpha}\eta_{ij}^{\beta}}{\sum_{k \in N_i} \tau_{ik}^{\alpha}\eta_{ik}^{\beta}}$$

- N_i: feasible next choices from state i
- α: importance of pheromone
- β: importance of heuristic information

Key perspective

More pheromone and shorter distance make an edge more attractive.



The state transition rule

$$p_{ij} = \frac{(\tau_{ij})^{\alpha} \cdot (\eta_{ij})^{\beta}}{\sum_{k \in A_i} (\tau_{ik})^{\alpha} \cdot (\eta_{ik})^{\beta}}$$

- A_i are cities reachable from i which have not yet been visited.
- α controls the importance of pheromones.
- β controls the importance of heuristic information.
- η_{ij} is the visibility, $1/d_{ij}$.

Reading the transition rule

- Large α makes ants trust collective history more strongly.
- Large β makes ants follow the built-in heuristic more aggressively.
- The choice remains probabilistic, which preserves exploration.

Design tension

ACO must balance **exploration of new solutions** with **exploitation of good components**.

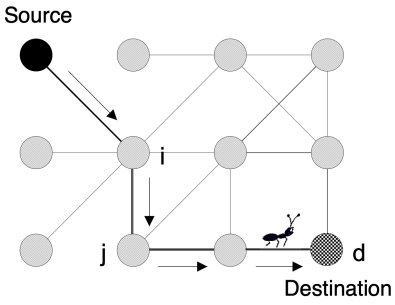
Pheromone update

After solutions are constructed, pheromone is updated as:

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \Delta\tau_{ij}$$

- ρ: evaporation rate
- $\Delta\tau_{ij}$: pheromone deposited proportional to quality, where the quality of the path is inversely proportional to the cost (for example distance). It is reinforcement from good solutions

- evaporation weakens stale choices
- reinforcement rewards useful choices



Pheromone update

After all ants have built a tour, pheromone trails are updated on all edges (i, j) as follows:

$$\tau_{ij} \leftarrow (1 - \rho)\tau_{ij} \qquad \forall (i, j) \qquad \text{evaporation}$$

$$\tau_{ij} \leftarrow \tau_{ij} + \Delta\tau_{ij} \qquad \forall (i, j) \qquad \text{pheromone deposit}$$

$$\text{where} \qquad \Delta\tau_{ij} = \sum_{k} \Delta\tau_{ij}^{k}$$

$$\Delta\tau_{ij}^{k} = \begin{cases} \dfrac{1}{L^k}, & \text{if ant } k \text{ used edge } (i, j), \\ 0, & \text{otherwise.} \end{cases}$$

Here, L^k is the length of the solution found by ant k.

Why evaporation matters

- Without evaporation, early random choices may dominate forever.
- Evaporation allows the colony to **forget** poor historical decisions.
- This is crucial when the search process needs to adapt.

Takeaway

Evaporation is not a minor detail; it is what prevents pheromone memory from becoming rigid.

Why ACO can work well

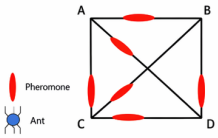
- Better solutions deposit more pheromone.
- Good components are reused across many candidate solutions.
- Over time, the population learns a biased distribution over promising decisions.

Intuition

ACO does not search by moving points around. It searches by **reweighting which decisions are likely to be made next**.

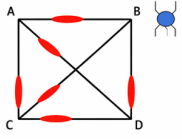
Running example: Traveling Salesman Problem (4-city TSP)

1 Initially, random levels of pheromone are scattered on the edges



AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

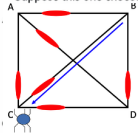
2 An ant is placed at a random node



AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

3 The ant decides where to go from that node, based on probabilities calculated from:
- pheromone strengths,
- next-hop distances.

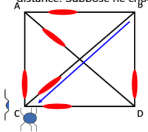
Suppose this one chooses BC



AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

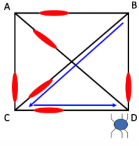
4 The ant is now at C, and has a 'tour memory' = {B, C} – so he cannot visit B or C again.

Again, he decides next hop (from those allowed) based on pheromone strength and distance: Suppose he chooses CD



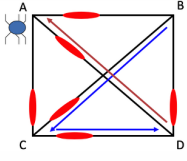
AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

5 The ant is now at D, and has a 'tour memory' = {B, C, D}
There is only one place he can go now:



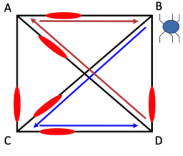
AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

6 So, he has nearly finished his tour, having gone over the links: BC, CD, and DA.



AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

7 So, he has nearly finished his tour, having gone over the links: BC, CD, and DA. AB is added to complete the round trip.



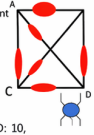
AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

8 Now, pheromone on the tour is increased, in line with the fitness of that tour.

Next, pheromone everywhere is decreased a little, to model decay of trail strength over time.

We start again, with another ant in a random position. Where will he go?

Next, the actual algorithm and variants.



AB: 20, AC: 30, AD: 10,
BC: 10, BD: 30, CD: 20

Edge — Pheromone (amount on edge) — Ant move / tour path

Distances (symmetric): AB=20, AC=30, AD=10, BC=10, BD=30, CD=20

How an ant builds a TSP tour

A tour

1. Start at an initial city.
2. Choose the next unvisited city using pheromone and distance information.
3. Record visited cities to avoid illegal revisits.
4. Continue until all cities have been visited.
5. Close the tour and evaluate its total length.

A simple TSP walkthrough

- Initially, pheromone is spread across edges.
- One ant samples a legal tour.
- A shorter tour produces stronger pheromone reinforcement.
- After many ants and many rounds, better tours become more likely.

The basic Ant System algorithm

1. Initialize pheromone values.
2. For each ant, construct a feasible solution.
3. Evaluate solution quality.
4. Evaporate pheromone globally.
5. Deposit pheromone based on selected solutions.
6. Repeat until a stopping criterion is met.

```
LOOP
    Place one ant on each city \* #_cities = #_ants \*
    FOR step := 1 to #_ants \* each ant builds a tour \*
        FOR k := 1 to #_cities \* each ant adds a city to its path \*
            Choose the next city to move to applying a
            probabilistic state transition rule
        END-FOR
    END-FOR
    Update pheromone trails
UNTIL End_condition
```

Observation

The algorithmic loop is simple, but the collective search distribution it induces can be quite rich.

Strengths and Weaknesses of ACO

Strengths

- well suited to combinatorial optimisation
- naturally incorporates domain heuristics
- distributed and adaptive
- effective when good solutions share reusable components

Weaknesses

- can be sensitive to parameter settings
- may stagnate if pheromone concentrates too early
- can be computationally expensive on large instances
- requires a well-designed construction representation

Takeaway

ACO is powerful, but its success depends on how well the problem is encoded as a constructive search process.

Applications of PSO and ACO

PSO

- Continuous optimisation
- Hyperparameter tuning
- Control and robotics
- Engineering design
- Feature selection and clustering

ACO

- Routing and shortest paths
- TSP and vehicle routing
- Scheduling and timetabling
- Network optimisation
- Path planning and logistics

Takeaway

PSO is most natural for continuous black-box optimisation, whereas ACO is most natural for discrete combinatorial optimisation.

A unifying summary

One idea to remember

Ant Colony Optimisation is a constructive search method in which a population of agents learns by writing useful information into the environment through **pheromone-based memory**.

- solutions are built incrementally
- good components are reinforced
- evaporation keeps the search adaptive